D3 / ROCKET MULTIVALUE

D3CODE Order Application

An order-to-stock application on the Crystal Logic D3 server, built, verified and pressure-tested through D3 Code. Modelled on the MerryPak order / stock / valuecard domain.

6	2	12	76
DATA FILES	INDEXES	PROGRAMS	PRESSURE CHECKS

16
RUNBOOK CHECKS

1. What it does

A customer places an order made of sale lines. Each line validates the customer, product, available stock and the customer's credit limit, takes an update lock on the product to decrement stock, prices the line, and accumulates the order subtotal, 15% VAT and total. Goods can be received, orders settled and cancelled, and goods returned as credit notes. Every stock movement is journalled so stock and balances can be reconciled.

> **Business rules.** VAT fixed at 15%; prices come from the product record; stock can never go negative; a sale that would exceed the credit limit is refused before any write; a paid order cannot be cancelled (credit it instead); cancelling an order whose product was removed reports `OK.PARTIAL` instead of silently skipping the stock restore.

2. Files and data model

File	Key	Attributes
`D3CODE.CUSTOMERS`	customer code	1 Name · 2 Email · 3 Credit limit · 4 Balance · 5 Terms · 6 Order list (multivalue)
`D3CODE.PRODUCTS`	SKU	1 Description · 2 Unit price · 3 Qty on hand · 4 Reorder level
`D3CODE.ORDERS`	order no	1 Date · 2 Customer · 3 Status · 4 Subtotal · 5 VAT · 6 Total · 7 Line count
`D3CODE.LINES`	`order*seq`	1 Order · 2 SKU · 3 Qty · 4 Price · 5 Line total
`D3CODE.MOVES`	`order*seq`	1 SKU · 2 Type (SALE/RECEIPT/CREDIT) · 3 Qty · 4 Reference · 5 Date

`D3CODE.PAYMENTS`	`order*P1`	1 Order · 2 Amount · 3 Date · 4 Type

Dictionary items are built by `D3CODE.DICTGEN`; AQL reserved words are avoided (the total field is `AMOUNT`, not `TOTAL`). Indexes: `D3CODE.PRODUCTS A1` (description) and `D3CODE.ORDERS A2` (customer).

3. Programs

Program	Type	Purpose
`D3CODE.DICTGEN`	main	Build every dictionary item (a BASIC writer, because the editor cannot express empty attributes).
`D3CODE.SEEDX` / `SEED`	sub / main	Baseline 4 customers and 5 products.
`D3CODE.RESETX`	subroutine	Empty every data file and re-seed, keeping dictionaries and indexes.
`D3CODE.SALE`	subroutine	`CALL D3CODE.SALE(CUST,SKU,QTY,ORDERNO,RESP)` — the posting logic.
`D3CODE.RECEIVE`	subroutine	`CALL D3CODE.RECEIVE(SKU,QTY,REF,RESP)` — goods in.
`D3CODE.CREDIT`	subroutine	`CALL D3CODE.CREDIT(ORDERNO,SKU,QTY,RESP)` — return goods, reduce the order and the balance.
`D3CODE.SETTLE`	subroutine	`CALL D3CODE.SETTLE(ORDERNO,AMOUNT,RESP)` — mark paid, reduce balance, journal a payment.
`D3CODE.CANCEL`	subroutine	`CALL D3CODE.CANCEL(ORDERNO,RESP)` — reverse an unpaid order.
`D3CODE.STATEMENT`	subroutine	`CALL D3CODE.STATEMENT(CUST,RESP)` — customer statement from the order list.
`D3CODE.BULK` / `RUN` / `REPORT` / `PROBE`	main	Load generator, worked example, low-stock report, file-open probe.
`D3CODE.PRESSURE`	main	Self-resetting scenario suite (26 in-D3 checks).

Return tokens from the posting subroutines

Token	Meaning
`<order>*<seq>`	Success — the order line written.
`NO.STOCK:<sku>`	Quantity exceeds quantity on hand — nothing written.
`CREDIT.LIMIT:<id>`	Balance + line total exceeds the limit — stock untouched.

`UNKNOWN.CUSTOMER` / `UNKNOWN.PRODUCT` / `UNKNOWN.ORDER`	Key not found.
`BAD.QTY`	Quantity below 1.
`ALREADY.PAID` / `ALREADY.CANCELLED` / `PAID.CANNOT.CANCEL` / `CANCELLED.ORDER`	State guards.
`OK.PARTIAL`	Cancelled, but stock could not be restored because a product was removed.
`FILE.*`	A required file could not be opened.

4. Build, verify, pressure, remove

```
# build files, dictionary, indexes, programs; run the worked example
node sandbox/d3-order-app/deploy.mjs --profile office

# reset, post once, assert the exact numbers (evidence/live-run.txt)
node sandbox/d3-order-app/verify.mjs  --profile office

# 26 in-D3 scenarios + concurrency, licence, volume and tool-path checks
node sandbox/d3-order-app/pressure.mjs --profile office

# delete every D3CODE.* file and program
node sandbox/d3-order-app/deploy.mjs --profile office --teardown
```

5. Verified results

Suite	Scope	Result
deploy	create files, write/compile/catalogue 16 programs, indexes, run	**73/73**
verify	worked example: totals, VAT, stock, indexes	**16/16**
pressure	boundaries, credit, locks, receipts, settle, cancel, credit notes, statements, bulk, reconciliation, missing files, 6-way and same-order contention, licence pressure, volume, index updates	**76/76**
d3code regression	harness live regression on the same account	**68/68**

Worked example recorded in D3 (`CT D3CODE.ORDERS B2001`): subtotal `694.85`, VAT `104.2275`, total `799.0775`, 2 lines. Concurrency: six buyers raced a three-unit product and exactly three won; five writers on one order produced exactly five lines and the correct subtotal; no deadlocks and no orphaned locks.

6. Everyday operations

Task	Command
Orders with totals	`LIST D3CODE.ORDERS CUSTOMER SUBTOTAL VAT AMOUNT (N`
Orders for a customer (index)	`LIST D3CODE.ORDERS WITH CUSTOMER = "C002" (N`
Low stock	`D3CODE.REPORT` or `SORT D3CODE.PRODUCTS BY QOH (N`
Stock journal	`LIST D3CODE.MOVES WITH SKU = "SKU-1" (N`
Customer statement	`CALL D3CODE.STATEMENT("C002",RESP)` from BASIC

> **Operational cautions.** Recreating a file (`DELETE-FILE` + `CREATE-FILE`) drops its dictionary and indexes — run `D3CODE.DICTGEN` and recreate the indexes afterwards. Sales, receipts, credits and settlements change stock and balances permanently: point tests at the `D3CODE.*` namespace only. D3 has no TCL `CALL` ; run a catalogued main program instead of calling a subroutine from the command line.

D3CODE Order Application runbook · built, verified and pressure-tested through D3 Code against Rocket D3 on `office.crystallogic.org` (account `dm`).
Companion: `docs/d3-live-regression.md` and the MerryPak ↔ D3 integration audit.